

Introduction to Class and Objects

1. User defined class:

A class is a blue print from which individual objects are created. It is the collection of data and methods that operate on data. It is declared by using 'class' keyword. Once defined, a class can be used to create objects of that type. Thus, a class is a *template* for an object. A general form of a **class** definition is shown here:

```
class classname {
    type instance-variable1;
    type instance-variable2;
    // ...
    type instance-variableN;
    type methodname1(parameter-list) {
    // body of method
    }
    type methodname2(parameter-list) {
    // body of method
    }
    // ...
    type methodnameN(parameter-list) {
    // body of method
    }
}
```

The data, or variables, defined within a **class** are called *instance variables*. The code is contained within *methods*. Collectively, the methods and variables defined within a class are called *members* of the class. The instance variables are acted upon and accessed by the methods defined for that class. Variables defined within a class are called instance variables because each instance of the class (that is, each object of the class) contains its own copy of these variables. Thus, the data for one object is separate and unique from the data for another.

Example:

```
class Box {
    double width;
    double height;
    double depth;
}
class BoxDemo {
    public static void main(String args[]) {
        Box mybox = new Box();
        double vol;
        mybox.width = 10;
        mybox.height = 20;
        mybox.depth = 15;
        vol = mybox.width * mybox.height * mybox.depth;
        System.out.println("Volume is " + vol);
    }
}
```

The output is:

Volume is 3000.0

Volume is 162.0

- ❖ **Declaring Objects:** An object is created from a class. In Java, the new key word is used to create new objects. There are three steps when creating an object from a class:
 - **Declaration:** A variable declaration with a variable name with an object type.
 - **Instantiation:** The 'new' key word is used to create the object.
 - **Initialization:** The 'new' keyword is followed by a call to a constructor. This call initializes the new object.

The **new** operator dynamically allocates memory for an object. It has this general form:

```
class-var = new classname( );
```

Here, *class-var* is a variable of the class type being created. The *classname* is the name of the class that is being instantiated. The class name followed by parentheses specifies the *constructor* for the class.

- ❖ **Introducing Methods:** Methods are the member of the class. The general form of a method is:

```
type name(parameter-list) {
    // body of method
}
```

Here, *type* specifies the type of data returned by the method. This can be any valid type, including class types that we create. If the method does not return a value, its return type must be **void**. The name of the method is specified by *name*. This can be any legal identifier other than those already used by other items within the current scope. The *parameter-list* is a sequence of type and identifier pairs separated by commas. Parameters are essentially variables that receive the value of the *arguments* passed to the method when it is called. If the method has no parameters, then the parameter list will be empty.

Methods that have a return type other than **void** return a value to the calling routine using the following form of the **return** statement:

```
return value;
```

Here, *value* is the value returned.

Example:

```
class Box {
    double width;
    double height;
    double depth;
    // display volume of a box
    void volume() {
        System.out.println("Volume is ");
        System.out.println(width * height * depth);
    }
}

class BoxDemo1 {
    public static void main(String args[]) {
        Box mybox = new Box();
    }
}
```

```

mybox.width = 10;
mybox.height = 20;
mybox.depth = 15;
mybox.volume();
}
}

```

Output:

Volume is 3000.0

The above example can also be illustrated by returning the result to the caller:

```

class Box {
    double width;
    double height;
    double depth;
    double volume() {
        return width * height * depth;
    }
}

class BoxDemo1 {
    public static void main(String args[]) {
        Box mybox1 = new Box();
        double vol;
        mybox1.width = 10;
        mybox1.height = 20;
        mybox1.depth = 15;
        vol = mybox1.volume();
        System.out.println("Volume is " + vol);
    }
}

```

When **volume()** is called the value returned by **volume()** is received by the variable 'vol'. There are two important things to understand about returning values:

- The type of data returned by a method must be compatible with the return type specified by the method. For example, if the return type of some method is **boolean**, we could not return an integer.
- The variable receiving the value returned by a method must also be compatible with the return type specified for the method.

❖ Adding a Method That Takes Parameters

```

class Box {
    double width;
    double height;
    double depth;
    double volume() {
        return width * height * depth;
    }
    void setDim(double w, double h, double d) {
        width = w;

```

```

height = h;
depth = d;
}
}
class BoxDemo1 {
public static void main(String args[]) {
Box mybox1 = new Box();
double vol;
mybox1.setDim(10, 20, 15);
vol = mybox1.volume();
System.out.println("Volume is " + vol);
}
}

```

Here the **setDim()** method is used to set the dimensions of each box. When `mybox1.setDim(10, 20, 15);` is executed, 10 is copied into parameter **w**, 20 is copied into **h**, and 15 is copied into **d**. Inside **setDim()** the values of **w**, **h**, and **d** are then assigned to **width**, **height**, and **depth**, respectively.

2. Constructors

Java allows objects to initialize themselves when they are created through the use of a constructor. A *constructor* initializes an object immediately upon creation. It has the same name as the class in which it resides and is syntactically similar to a method. Once defined, the constructor is automatically called immediately after the object is created, before the **new** operator completes. Constructors have no return type, not even **void**. This is because the implicit return type of a class' constructor is the class type itself. Example of constructor is:

```

public class Puppy{
    int puppyAge;
    public Puppy(String name){
        // This constructor has one parameter, name.
        System.out.println("Passed Name is :"+ name );
    }
    public void setAge( int age ){
        puppyAge = age;
    }
    public void getAge( ){
        System.out.println("Puppy's age is :"+ puppyAge );
    }
    public static void main(String []args){
        Puppy myPuppy = new Puppy( "tommy" );
        myPuppy.setAge( 2 );
        myPuppy.getAge( );
    }
}

```

If we compile and run the above program, then it would produce the following result:

```
Passed Name is :tommy
```

Puppy's age is :2

3. Packages:

The package is both a naming and a visibility control mechanism. We can define classes inside a package that are not accessible by code outside that package. Packages are the Java's way of grouping a variety of classes together. The main reason for using packages is to guarantee the uniqueness of class names.

❖ Finding Packages and CLASSPATH

We can find the packages that we have created in three ways. First, by default, the Java run-time system uses the current working directory as its starting point. Thus, if our package is in a subdirectory of the current directory, it will be found. Second, we can specify a directory path or paths by setting the **CLASSPATH** environmental variable. Third, we can use the **-classpath** option with **java** and **javac** to specify the path to our classes. For example, consider the following package specification:

```
package MyPack
```

To find **MyPack**, one of three things must be true. Either the program can be executed from a directory immediately above **MyPack**, or the **CLASSPATH** must be set to include the path to **MyPack**, or the **-classpath** option must specify the path to **MyPack** when the program is run via **java**. When the second two options are used, the class path *must not* include **MyPack**, itself. It must simply specify the *path to MyPack*. For example, in a Windows environment, if the path to **MyPack** is;

```
C:\MyPrograms\Java\MyPack
```

Then the class path to **MyPack** is;

```
C:\MyPrograms\Java
```

❖ Defining a Package

To create a package, we use the **'package'** command as the first statement in a Java source file. Any classes declared within that file will belong to the specified package. If we omit the **package** statement, the class names are put into the default package, which has no name. The general form of the **package** statement:

```
package packagename;
```

❖ Importing Packages

Java includes the **import** statement to bring certain classes, or entire packages, into visibility. The general form of the **import** statement:

```
import packagename.*;
```

Example:

```
package MyPack;
public class Balance
{
    String name;
    double bal;
    public Balance(String n, double b)
    {
        name = n;
        bal = b;
    }
}
```

```

    }
    public void show()
    {
        System.out.println(name + ":" + bal);
    }
}

import MyPack.*;
class TestBalance
{
    public static void main(String args[])
    {
        Balance test = new Balance("Hari", 1000.3);
        test.show();
    }
}

```

❖ Creating subpackage

To create a subpackage the syntax is:

```
package packagename.subpackagename.*;
```

4. Inheritance

Inheritance is the process by which a class inherits the property of another class. The class which is being inherited is known as the super class (or base class) and the class that inherits the property is known sub class (or derived class). To inherit the content of superclass, the base class uses the **extends** keyword. Java provides multilevel inheritance but does not support multiple inheritance.

Example:

```

class A
{
    int i, j;
    void showij()
    {
        System.out.println("i and j: " + i + " " + j);
    }
}

class B extends A
{
    int k;
    void showk()
    {
        System.out.println("k: " + k);
    }
    void sum()
    {
        System.out.println("i+j+k: " + (i+j+k));
    }
}

```

```

    }
    }

    class SimpleInheritance
    {
    public static void main(String args[])
    {
    B subOb = new B();
    subOb.i = 7;
    subOb.j = 8;
    subOb.k = 9;
    System.out.println("Contents of subOb: ");
    subOb.showij();
    subOb.showk();
    System.out.println("Sum of i, j and k in subOb:");
    subOb.sum();
    }
    }

```

The output from this program is shown here:

Contents of subOb:

i and j: 7 8

k: 9

Sum of i, j and k in subOb:

i+j+k: 24

5. Interfaces

Using the keyword **interface**, we can specify what a class must do, but not how it does it. Interfaces

are syntactically similar to classes, but they lack instance variables, and their methods are declared without any body. Once it is defined, any number of classes can implement an **interface**. Also, one class can implement any number of interfaces. To implement an interface, a class must create the complete set of methods defined by the interface. However, each class is free to determine the details of its own implementation.

❖ Defining an Interface

An interface is defined much like a class. The general form of an interface:

```

interface interfacename
{
    return-type method-name1(parameter-list);
    return-type method-name2(parameter-list);
    type varname1 = value;
    type varname2 = value;
    // ...
    return-type method-nameN(parameter-list);
    type varnameN = value;
}

```

interfacename is the name of the interface, and can be any valid identifier. The methods that are declared have no bodies. They end with a semicolon after the parameter list. Each class that includes an interface must implement all of the methods. Variables can be declared inside of interface declarations. All methods and variables are implicitly **public**.

Example:

```
interface Callback
{
void callback(int param);
}
```

❖ Implementing Interfaces

Once an **interface** has been defined, one or more classes can implement that interface. To implement an interface, include the **implements** clause in a class definition, and then create the methods defined by the interface. The general form of a class that includes the **implements** clause is:

```
class classname implements interface
{
// class-body
}
```

If a class implements more than one interface, the interfaces are separated with a comma. The methods that implement an interface must be declared **public**.

Here is a small example class that implements the **Callback** interface shown earlier.

```
class Client implements Callback
{
public void callback(int p)
{
System.out.println("callback called with " + p);
}
void nonIfaceMeth()
{
System.out.println("Classes that implement interfaces " +
"may also define other members, too.");
}
}
```

It is both permissible and common for classes that implement interfaces to define additional members of their own. For example, the following version of **Client** implements **callback()** and adds the method **nonIfaceMeth()**:

The following example calls the **callback()** method via an interface reference variable:

```
class TestIface
{
public static void main(String args[])
{
Client c = new Client();
```



```
c.callback(42);
}
}
```

The output of this program is shown here:
callback called with 42

6. Inner Classes

It is the class defined inside another class. It has access to all of the variables and methods of its outer class and may refer to them directly. But the reverse is not true i.e the outer class cannot have direct access to all the data members and function of the inner class. The following program illustrates how to define and use an inner class. The class named **Outer** has one instance variable named **outer_x**, one instance method named **test()**, and defines one inner class called **Inner**.

```
class Outer
{
    int outer_x = 100;
    void test()
    {
        Inner inner = new Inner();
        inner.display();
    }
    class Inner
    {
        void display()
        {
            System.out.println("display: outer_x = " + outer_x);
        }
    }
}
class InnerClassDemo
{
    public static void main(String args[])
    {
        Outer outer = new Outer();
        outer.test();
    }
}
```

Output from this application is shown here:
display: outer_x = 100

7. Access Protection Mechanism

Java's access specifiers are **public**, **private**, and **protected**. Java also defines a default access level. **protected** applies only when inheritance is involved. When a member of a class is modified by the **public** specifier, then that member can be accessed by any other code. When a member of a class is specified as **private**, then that member can only be accessed by other

members of its class. When no access specifier is used, then by default the member of a class is public within its own package, but cannot be accessed outside of its package.

Example:

```
class Test
{
int a; // default access
public int b; // public access
private int c; // private access
void setc(int i)
{
c = i;
}
int getc()
{
return c;
}
}
```

```
class AccessTest
{
public static void main(String args[])
{
Test ob = new Test();
// These are OK, a and b may be accessed directly
ob.a = 10;
ob.b = 20;
ob.c = 100; // Error!
ob.setc(100); // OK
System.out.println("a, b, and c: " + ob.a + " " + ob.b + " " + ob.getc());
}
}
```